[NAME]: GAYATHRI S

[STUDENT_ID]:  (26007F_012)

[PROJECT TITLE]: RED-BLACK TREE

[DATE]:14/10/2026

---

---[DESCRIPTION]---

[Introduction]

A Red-Black tree is a self-Balancing Binary Search tree.

Every node is coloured either red or black, and it follows four main properties to stay balanced.

=>The root is always Black
=>No two red Nodes can be adjacent (a Red node children bust be Black
=>Every path from a node to its descendant NULL leaves contains the same number of Black nodes
=>All leaves are (NIL) are Black

---

[Objectives]

=>To study limitation of normal binary search tree and why balancing is needed
=>To implement the red-black tree properties(red/black coloring and black - balancing
=>to implement core operation like insertion, deletion, search, rotation
=>To visualization how tree is balanced after each operation
=>To compare its performance with AVL tree and normal BST

---

[Working principle]

1.Insert as BST
	new node is always inserted like normal BST and coloured as BLACK

2.Root is Black
	if tree is empty ,new node becomes changed root node and is changed to Black

3.check violation
	if new parent is black = no problem
	if parent is RED violation occurs
	case 1 : siblings are RED = do recolouring , parent & siblings BLACK , Grandparent RED
	case 2:siblings is BLACK = do rotation and recolouring

**4.maintain balancing**
**After every operation , every path from root to leaf have same balance number of BLACK**

**5.Deletion**
**if black node is deleted , black height is disturbed.it creates double block and is fixed by rotation and recolouring**

**6.result**
**tree remains balanced**

---

---

**[Functions]**

**1.INSERTION**
**insert new node as RED like BST, then fix violations using recolouring and rotations**

**2.DELETION**
**delete the nodes, if deleted node is BLACK, fix the double black violation**

**3.LEFT ROTATION**
**Rotate the subtree to left to balance when right is heavy**

**4.RIGHT ROTATION**
**Rotate the subtree to right to balance when left is heavy**

**5.SEARCH**
**search a key like BST(0logn), tree is balanced**

**6.RECOLOUR**
**change colour of node from RED TO BLACK or BLACK TO RED to maintain process**

**7.GRANDPARENT/SIBLINGS**
**helper functions to find relatives of node to decide which case to apply for fixing**

**8.DISPLAY/INORDER TRAVERSAL**
**Traverse and display the tree with node colour and structure**

---

---

**[File Structure]**

**=>Main.c**

**=>Red_Black_tree.h**

=>Red_Black_tree.c

=>opr.c

---

[Advantages]

=>Always balanced

=>Fast operations

=>used in real world - map,set,multimap

=>less memory

---

[Conclusion]

Red-Black tree is efficient self-balancing that maintains 0(logn) complexity by using colour properties, recolouring and rotations.
it overcomes the drawback of BST and is faster than AVL for insertion/deletion.it is widely used in real-world systems like C++ STL and Databases.